

Internet of Things (IoT)
/
Internet des Objets (IdO)
/
Physical Protocols : WiFi

Gilles Menez
University Nice Côte d'Azur
email : menez@unice.fr
www : www.i3s.unice.fr/~menez

October 13, 2025: V 1.0

What you should learn ...

1. We "plug" the ESP32 on WiFi !

Two remarks

1. When the code is given ... YOU ARE SUPPOSED TO study IT !
and not only to make it run ... I already KNOW it is running ;-)
2. Sections titled "TODO : Votre travail !" explain what you have to experiment or create by yourself from examples.

Do not hesitate to be proactive and to propose improvements of questions/answers/requirements !

Table of Contents

1	Few Features of WiFi in ESP context	5
1.1	WiFi infrastructure by ESP	5
1.2	Security Modes	5
1.3	Scanning neighborhood	5
1.4	Promiscuous mode	6
1.5	2.4 Ghz !!	6
2	Some examples, some codes	7
2.1	Connect to an AP	7
2.1.1	WiFi Status	10
2.2	When AP is not statically defined	11
2.3	The ESP is an AP	14
2.4	TODO : WiFi	14
2.5	Static/Fixed IP Addresses	15
2.6	For the future : ESP-MESH	15
3	Over The Air : OTA	16
3.0.1	References	16
3.1	To be able to switch	16
3.2	An "OTA enabled" sketch	16
3.2.1	The user "view" for enabling OTA	17
3.2.2	OTA configuration	18
4	Uploading OTA	21
4.1	Uploading from Arduino IDE	21
4.2	Uploading from the Shell	22
4.3	Goodbye ... serialxxxx	22
4.4	TODO : OTA	22

List of Figures

1 Few Features of WiFi in ESP context

The WiFi libraries (ESP-IDF) provide support for configuring and monitoring the ESP32 WiFi networking functional block :

- https://docs.espressif.com/projects/esp-idf/en/latest/esp32/api-reference/network/esp_wifi.html
- <https://github.com/espressif/arduino-esp32/tree/master/libraries/WiFi/examples>

We use this interface which is an **encapsulation of Arduino IDE** !

- <https://www.arduino.cc/en/Reference/WiFi>

1.1 WiFi infrastructure by ESP

ESP can be deployed in several configurations :

- "Station mode" (aka "STA mode" or "WiFi client mode") :
ESP32 connects to an access point ... like your laptop connects to your box in your home.
- "AP mode" (aka "Soft-AP mode" or "Access Point mode"):
Other stations connect to the ESP32 ... the ESP is the box !
- "Combined AP-STA mode" :
ESP32 is concurrently an access point and a station connected to another access point
Some libraries use this feature to emulate a "Grid WIFI"

1.2 Security Modes

Various security modes for the above (WPA, WPA2, WEP, etc.) can be adopted.

1.3 Scanning neighborhood

Scanning for access points (active & passive scanning) is a major feature for objects.

- The reason for client scanning is **to determine a suitable AP** to which the client may need to roam now or in the future.

A ESP client can use two scanning methods : **active and passive**.

- During an **active scan**, the client radio transmits a probe request and listens for a probe response from an AP.
- With a **passive scan**, the client radio listens on each channel for beacons sent periodically by an AP.

A passive scan generally takes more time, since the client must listen and wait for a beacon versus actively probing to find an AP.

Another limitation with a passive scan is that if the client does not wait long enough on a channel, then the client may miss an AP beacon.

1.4 Promiscuous mode

ESP allows to run in "Promiscuous mode" for monitoring of IEEE802.11 WiFi packets.

- In computer networking, "promiscuous mode" is a mode for a wired network interface controller (NIC) or wireless network interface controller (WNIC) **that causes the controller to pass all traffic it receives to the central processing unit (CPU)** rather than passing only the frames that the controller is specifically programmed to receive.

Since CPU can see packets ... it can analyze and more ...

- This mode is used for packet sniffing !

And ESP is often used for that :-)

1.5 2.4 Ghz !!

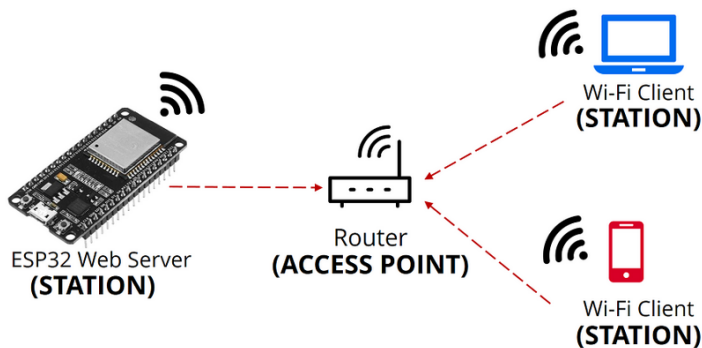
Your ESP EVM can only use 2.4 Ghz (Band) WiFi !

2 Some examples, some codes

2.1 Connect to an AP

Dans un premier temps, il s'agit de connecter notre ESP à un réseau physique WiFi pour pouvoir envoyer des données avec notre application et le protocole HTTP.

Dans ce contexte, selon la terminologie WiFi, l'ESP est une "Station" (STA) qui se connecte à un "Access Point" (AP).



L'Access Point ne peut pas être un de ceux de l'UniCA pour plein de raisons bien tristes de sécurité/authentification/filtrage/volonté.

- L'UniCA ne supporte donc pas trop l'IoT :-)
- Donc l'AP, sera sans doute votre Smartphone ou le réseau IOT/iotmiage.

Le sketch qui suit illustre comment l'ESP peut se connecter à un Access Point (AP) d'un SSID.

- On utilise la bibliothèque WiFi de l'arduino : <https://www.arduino.cc/en/Reference/WiFi>
dont on sait qu'elle "wrap" les primitives ESP-IDF : <https://github.com/espressif/arduino-esp32/tree/master/libraries>

```
1  /** Basic/Static Wifi connection
2      Fichier wificonnect_basic/wificonnect_basic.ino */
3  #include <WiFi.h> // https://www.arduino.cc/en/Reference/WiFi
4  #include "wifi_utils.h"
5  #define USE_SERIAL Serial
6  /*----- Arduino IDE paradigm : setup+loop -----*/
7  void setup(){
8      Serial.begin(9600); /* Serial connection -----*/
9      while(!Serial); //wait for a serial connection
10
11      /* WiFi -----*/
12      String hostname = "Mon petit objet ESP32";
13      String ssid = String("Bbox-EA6814CF");
14      String passwd = String("Hvtx6bGfD4Sn7PCs2r");
15
16      /* Connection from a SSID -----*/
17      wificonnect_basic(hostname, ssid, passwd);
18
19      /* WiFi status -----*/
20      if (WiFi.status() == WL_CONNECTED){
21          USE_SERIAL.print("\nWiFi connected : yes ! \n");
```

```

22     wifi_printstatus(0);
23 }
24 else {
25     USE_SERIAL.print("\nWiFi connected : no ! \n");
26     // ESP.restart();
27 }
28 }
29
30 void loop(){
31     // no code
32     // USE_SERIAL.println(".");
33     // WiFi.disconnect(); // at the end !
34 }

```

```

1  /** Basic/Static Wifi connection
2
3      Fichier wificonnect_basic/wifi_utils.h */
4
5  #include <WiFi.h> // https://www.arduino.cc/en/Reference/WiFi
6  #include <WiFiMulti.h>
7
8  #define WiFiMaxTry 10
9  #define SaveDisconnectTime 1000 // Connection may need several tries
10                                     // Time in ms for save disconnection, => delay between try
11                                     // cf https://github.com/espressif/arduino-esp32/issues/2501
12
13 void wifi_printstatus();
14 void wificonnect_basic(String hostname, String ssid, String passwd);

```

```

1  /** Basic/Static Wifi connection
2      Fichier wificonnect_basic/wifi_utils.ino */
3
4  #include <WiFi.h> // https://www.arduino.cc/en/Reference/WiFi
5  #include "wifi_utils.h"
6  String translateEncryptionType(wifi_auth_mode_t encryptionType);
7
8  #define USE_SERIAL Serial
9
10 void wificonnect_basic(String hostname, String ssid, String passwd){
11     int nbtry = 0; // Nb of try to connect
12
13     WiFi.mode(WIFI_OFF);
14     WiFi.mode(WIFI_STA); // Set WiFi to station mode
15     // delete old config
16     // WiFi.config(INADDR_NONE, INADDR_NONE, INADDR_NONE, INADDR_NONE);
17     WiFi.disconnect(true); // Disconnect from an AP if it was previously connected
18     // WiFi.persistent(false); // Avoid to store Wifi configuration in Flash
19
20     // Define hostname before begin => in C str ! not C++
21     WiFi.setHostname(hostname.c_str());
22     USE_SERIAL.printf("\nAttempting %d to connect AP of SSID : %s", nbtry, ssid.c_str());
23     WiFi.begin(ssid.c_str(), passwd.c_str());
24     //WiFi.begin(ssid.c_str(), passwd.c_str(), 0, WiFi.BSSID(thegoodone));
25
26     while(WiFi.status() != WL_CONNECTED && (nbtry < WiFiMaxTry)){
27         delay(SaveDisconnectTime); // 500ms seems to work in most cases, may depend on AP
28         USE_SERIAL.print(".");
29         nbtry++;
30     }
31     if (nbtry == WiFiMaxTry)
32         ESP.restart();
33 }
34 /*-----*/
35 String translateEncryptionType(wifi_auth_mode_t encryptionType) {

```



```

36 // encryptiontype to string
37 // cf https://www.arduino.cc/en/Reference/WiFiEncryptionType
38 switch (encryptionType) {
39 case (WIFI_AUTH_OPEN):
40     return "Open";
41 case (WIFI_AUTH_WEP):
42     return "WEP";
43 case (WIFI_AUTH_WPA_PSK):
44     return "WPA_PSK";
45 case (WIFI_AUTH_WPA2_PSK):
46     return "WPA2_PSK";
47 case (WIFI_AUTH_WPA_WPA2_PSK):
48     return "WPA_WPA2_PSK";
49 case (WIFI_AUTH_WPA2_ENTERPRISE):
50     return "WPA2_ENTERPRISE";
51 }
52 }
53 /*-----*/
54 void wifi_printstatus(int C){
55     /* print the status of the connected wifi in two ways ! */
56
57     if (C){
58         // Use Pure C => array of chars
59         USE_SERIAL.printf("WiFi Status : \n");
60         USE_SERIAL.printf("\tIP address : %s\n", WiFi.localIP().toString().c_str());
61         USE_SERIAL.printf("\tMAC address : %s\n", WiFi.macAddress().c_str());
62         USE_SERIAL.printf("\tSSID : %s\n", WiFi.SSID());
63         USE_SERIAL.printf("\tReceived Signal Strength Indication : %ld dBm\n",WiFi.RSSI());
64         USE_SERIAL.printf("\tReceived Signal Strength Indication : %ld %\n",
65             constrain(2 * (WiFi.RSSI() + 100), 0, 100));
66         USE_SERIAL.printf("\tBSSID : %s\n", WiFi.BSSIDstr().c_str());
67         USE_SERIAL.printf("\tEncryption type : %s\n", translateEncryptionType(WiFi.encryptionType(0)));
68     }
69     else {
70         // Use of C++ => String !
71         String s = "WiFi Status : \n";
72         //s += "\t#" + String() + "\n";
73         s += "\tIP address : " + WiFi.localIP().toString() + "\n";
74         s += "\tMAC address : " + String(WiFi.macAddress()) + "\n";
75         s += "\tSSID : " + String(WiFi.SSID()) + "\n";
76         s += "\tReceived Sig Strength Indication : " + String(WiFi.RSSI()) + " dBm\n";
77         s += "\tReceived Sig Strength Indication : " +
78             String(constrain(2 * (WiFi.RSSI() + 100), 0, 100)) + " %\n";
79         s += "\tBSSID : " + String(WiFi.BSSIDstr()) + "\n";
80         s += "\tEncryption type : " + translateEncryptionType(WiFi.encryptionType(0)) + "\n";
81         USE_SERIAL.print(s);
82     }
83 }
84

```

Dans "cette version", le code de l'ESP connaît statiquement les "coordonnées" du réseau physique. Il sait

- le SSID
- et le crédentiel

Puisque la fonction `wificonnect_basic` connaît statiquement le SSID du réseau souhaité, elle peut l'utiliser lorsqu'elle appelle "WiFi.begin".

1. On place l'ESP en mode station : STA mode or WiFi client mode : `WiFi.mode(WIFI_STA)`

Explicitly set the ESP to be a WiFi-client, otherwise, it by default, would try to act as both a client and an access-point and could cause network-issues with your other WiFi-devices on your WiFi-network.

BE CAREFUL ! because **SSID of WIFI networks ARE NOT SUPPORTED IF they contain whitespaces !**

2. On donne les "credentials" pour accéder au réseau (SSID + MDP) et on se connecte au réseau : `WiFi.begin(...)`

3. On attend que la connexion soit effective en testant le status : `WiFi.status()`

On essaye un certain nombre de fois.

Cette approche est "la base" mais il n'est pas évident de connaître statiquement avec précision le SSID du réseau sur lequel doit se connecter l'objet car la plupart du temps on ne sait pas a priori où il sera déployé.

2.1.1 WiFi Status

Vous remarquez la fonction "wifi_printstatus" qui donne, une fois la connexion établie, les caractéristiques de ce lien.

```
20:25:27.353 -> Attempting 0 to connect AP of SSID : Bbox-EA6814CF.
20:25:28.350 -> WiFi connected : yes !
20:25:28.350 -> WiFi Status :
20:25:28.382 ->         IP address : 192.168.1.69
20:25:28.415 ->         MAC address : AC:67:B2:38:67:E4
20:25:28.446 ->         SSID : Bbox-EA6814CF
20:25:28.479 ->         Received Sig Strength Indication : -60 dBm
20:25:28.510 ->         Received Sig Strength Indication : 80 %
20:25:28.543 ->         BSSID : BA:C4:5B:7E:70:B5
20:25:28.576 ->         Encryption type : Open
```

2.2 When AP is not statically defined

Dans le code qui suit, l'ESP est toujours une STA qui se connecte à un AP mais il va pouvoir choisir l'AP parmi une liste statique de points d'accès définie dans le code.

- Il prendra celui qui a le meilleur RSSI.

```
1  /** Basic/Static Wifi connection
2      Fichier wificonnect_simple/wificonnect_simple.ino */
3  #include <WiFi.h> // https://www.arduino.cc/en/Reference/WiFi
4  #include "wifi_utils.h"
5  #define USE_SERIAL Serial
6  /*----- Arduino IDE paradigm : setup+loop -----*/
7  void setup(){
8      Serial.begin(9600); /* Serial connection -----*/
9      while(!Serial); //wait for a serial connection
10
11     /* WiFi -----*/
12     String hostname = "Mon petit objet ESP32";
13
14     /* Connection from a list of SSID */
15     wificonnect_multi(hostname);
16
17     /* WiFi status -----*/
18     if (WiFi.status() == WL_CONNECTED){
19         USE_SERIAL.print("\nWiFi connected : yes ! \n");
20         wifi_printstatus(0);
21     }
22     else {
23         USE_SERIAL.print("\nWiFi connected : no ! \n");
24         // ESP.restart();
25     }
26 }
27
28 void loop(){
29     // no code
30
31     // WiFi.disconnect(); // at the end !
32 }
```

```
1  /** Multi Wifi connection
2      Fichier wificonnect_multi/wifi_utils.h */
3
4  #include <WiFi.h> // https://www.arduino.cc/en/Reference/WiFi
5  #include <WiFiMulti.h>
6
7  #define WiFiMaxTry 10
8  #define SaveDisconnectTime 1000 // Connection may need several tries
9      // Time in ms for save disconnection, => delay between try
10     // cf https://github.com/espressif/arduino-esp32/issues/2501
11
12 void wifi_printstatus();
13 void wificonnect_multi(String hostname);
```

```
1  /** Multi Wifi connection
2      Fichier wificonnect_multi/wifi_utils.ino */
3
4  #include <WiFi.h> // https://www.arduino.cc/en/Reference/WiFi
5  #include "wifi_utils.h"
6
7  #define USE_SERIAL Serial
8
9  /*-----*/
```

```

10 void wificonnect_multi(String hostname){
11     int nbtry = 0; // Nb of try to connect
12
13     WiFiMulti wm; // Creates an instance of the WiFiMulti class
14     // Attention ! PAS arrivé à sortir l'instance de la fonction => heap error ! Why ???
15     wm.addAP("HUAWEI-6EC2", "FGY9MLBL");
16     wm.addAP("HUAWEI-553A", "QTM06RTT");
17     wm.addAP("GMAP", "vijx47050");
18     wm.addAP("IOT", "iotmiage");
19     wm.addAP("Bbox-EA6814CF", "Hvtx6bGfD4Sn7PCs2r");
20
21     //WiFi.mode(WIFI_OFF);
22     WiFi.mode(WIFI_STA); // Set WiFi to station mode
23     // delete old config
24     // WiFi.config(INADDR_NONE, INADDR_NONE, INADDR_NONE, INADDR_NONE);
25     WiFi.disconnect(true); // Disconnect from an AP if it was previously connected
26     // WiFi.persistent(false); // Avoid to store Wifi configuration in Flash
27
28     // Define hostname => in C str ! not C++
29     WiFi.setHostname(hostname.c_str());
30
31     while(wm.run() != WL_CONNECTED && (nbtry < WiFiMaxTry)) {
32         USE_SERIAL.printf("\nAttempting %d to connect AP", nbtry);
33         delay(SaveDisconnectTime);
34         USE_SERIAL.print(".");
35         nbtry++;
36     }
37
38     if(wm.run() == WL_CONNECTED) {
39         USE_SERIAL.printf("\nWifi Multi connected on %s !", WiFi.SSID());
40         wifi_printstatus(0);
41     }
42     else
43         ESP.restart();
44 }
45 /*-----*/
46 String translateEncryptionType(wifi_auth_mode_t encryptionType) {
47     // encryptiontype to string
48     // cf https://www.arduino.cc/en/Reference/WiFiEncryptionType
49     switch (encryptionType) {
50     case (WIFI_AUTH_OPEN):
51         return "Open";
52     case (WIFI_AUTH_WEP):
53         return "WEP";
54     case (WIFI_AUTH_WPA_PSK):
55         return "WPA_PSK";
56     case (WIFI_AUTH_WPA2_PSK):
57         return "WPA2_PSK";
58     case (WIFI_AUTH_WPA_WPA2_PSK):
59         return "WPA_WPA2_PSK";
60     case (WIFI_AUTH_WPA2_ENTERPRISE):
61         return "WPA2_ENTERPRISE";
62     }
63 }
64 /*-----*/
65 void wifi_printstatus(int C){
66     /* print the status of the connected wifi in two ways ! */
67
68     if (C){
69         // Use Pure C => array of chars
70         USE_SERIAL.printf("WiFi Status : \n");
71         USE_SERIAL.printf("\tIP address : %s\n", WiFi.localIP().toString().c_str());
72         USE_SERIAL.printf("\tMAC address : %s\n", WiFi.macAddress().c_str());
73         USE_SERIAL.printf("\tSSID : %s\n", WiFi.SSID());
74         USE_SERIAL.printf("\tReceived Signal Strength Indication : %ld dBm\n", WiFi.RSSI());
75         USE_SERIAL.printf("\tReceived Signal Strength Indication : %ld %\n",
76             constrain(2 * (WiFi.RSSI() + 100), 0, 100));

```

```

77     USE_SERIAL.printf("\tBSSID : %s\n", WiFi.BSSIDstr().c_str());
78     USE_SERIAL.printf("\tEncryption type : %s\n", translateEncryptionType(WiFi.encryptionType(0)));
79 }
80 else {
81     // Use of C++ => String !
82     String s = "WiFi Status : \n";
83     //s += "\t#" + String() + "\n";
84     s += "\tIP address : " + WiFi.localIP().toString() + "\n";
85     s += "\tMAC address : " + String(WiFi.macAddress()) + "\n";
86     s += "\tSSID : " + String(WiFi.SSID()) + "\n";
87     s += "\tReceived Sig Strength Indication : " + String(WiFi.RSSI()) + " dBm\n";
88     s += "\tReceived Sig Strength Indication : " +
89         String(constrain(2 * (WiFi.RSSI() + 100), 0, 100)) + " %\n";
90     s += "\tBSSID : " + String(WiFi.BSSIDstr()) + "\n";
91     s += "\tEncryption type : " + translateEncryptionType(WiFi.encryptionType(0)) + "\n";
92     USE_SERIAL.print(s);
93 }
94 }

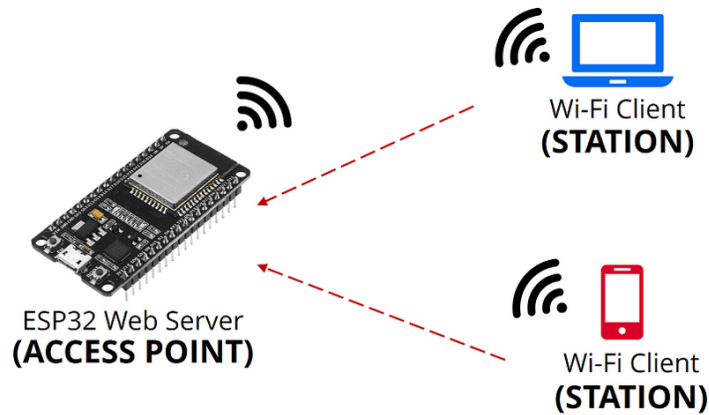
```

La "seule" différence avec la version précédente se situe dans la fonction `wificonnect_multi` qui travaille sur une liste d'AP potentiels.

- Elle nécessite l'inclusion du header file : `WiFiMulti.h`

2.3 The ESP is an AP

Faute de temps on ne l'utilisera pas dans cette UE, mais il faut savoir que l'ESP peut être configuré comme un AP et donc devenir l'"équivalent" de votre box internet.



- On peut alors avoir une connectivité même si il n'y a pas de "box" ou d'équipement existant.

C'est l'ESP qui jouera ce rôle.

2.4 TODO : WiFi

You have to experiment this code and understand ...

- Most of the time we will use `wifioconnect_multi` in our futures applications.
Just add your APs ...

2.5 Static/Fixed IP Addresses

If you need so set "manually" static/fixed IP Addresses for your ESP STA:

- <https://randomnerdtutorials.com/esp8266-nodemcu-static-fixed-ip-address-arduino/>
- <https://www.raspberryme.com/esp8266-nodemcu-adresse-ip-statique-fixe-arduino-ide/>

2.6 For the future : ESP-MESH

<https://randomnerdtutorials.com/esp-mesh-esp32-esp8266-painlessmesh/>

3 Over The Air : OTA

Le téléchargement du firmware sur la liaison série USB est possible pendant la phase de développement, lorsque l'objet est "proche" de votre station de développement.

Mais une fois que votre projet est terminé, et que vous le déployez dans un boîtier sur un arbre, il n'est pas si facile de télécharger des mises à jour avec des corrections de bogues ou de nouvelles fonctionnalités.

Une solution à ce problème est la mise à jour "Over The Air" (OTA) :

- Comme son nom l'indique, cette technologie vous permet de télécharger un nouveau code par WiFi ... sans fil.

3.0.1 References

- <https://docs.espressif.com/projects/esp-idf/en/latest/esp32/api-reference/system/ota.html>
- <https://github.com/espressif/esp-idf/tree/5c1044d84d/examples/system/ota>
- https://arduino-esp8266.readthedocs.io/en/latest/ota_updates/readme.html
- <https://projetsdiy.fr/arduinoota-esp32-mise-jour-sans-fil-wifi-ota-ide-arduino/>

3.1 To be able to switch ...

Il existe deux façons de mettre en œuvre la fonctionnalité OTA dans ESP32.

1. OTA de base : Les mises à jour OTA sont envoyées via l'IDE Arduino.
2. Web Updater OTA : Les mises à jour OTA sont envoyées via un navigateur Web.

L'inconvénient majeur d'une liaison sans fil est sa fragilité.

- On ne peut donc pas "oublier" l'ancien firmware avant d'avoir la garantie que le nouveau est correctement chargé.

Donc, pour implémenter OTA on a besoin d'une mémoire flash dont la capacité est d'au moins deux fois la taille de votre croquis :

1. OTA va permettre de charger une nouvelle version du firmware dans une partition inutilisée.
2. Puis, par une commutation de partitions, activer la version nouvellement chargée.

Le basculement de version n'ayant lieu que si le téléchargement est complet/parfait.

3.2 An "OTA enabled" sketch

Au niveau du code, un "ESP sous OTA" a tout simplement activé ce mécanisme dans son setup et dans sa boucle où il surveille l'occurrence d'une requête d'OTA:

3.2.1 The user "view" for enabling OTA

L'activation du mécanisme OTA se fait avec la fonction "setupOTA".

- La fonction correspondante est définie dans le fichier "ota.ino"

Attention aussi à la "non" compatibilité avec la fonction delay(). On préfère utiliser la fonction "millis()" dans "DoStmg()" pour afficher périodiquement l'uptime de l'ESP.

```
1  #include "wifi_utils.h"
2  #include "ota.h"
3  #include <uptime.h>
4  #include <RemoteDebug.h>
5
6  #define USE_SERIAL Serial
7
8  RemoteDebug Debug;
9
10 void setup() {
11     /* Serial connection -----*/
12     Serial.begin(9600);
13     while(!Serial); //wait for a serial connection
14
15     /* WiFi connection -----*/
16     String hostname = "Mon petit objet ESP32";
17     wifi_connect_multi(hostname);
18
19     /* WiFi status -----*/
20     if (WiFi.status() == WL_CONNECTED){
21         USE_SERIAL.print("\nWiFi connected : yes ! \n");
22         wifi_printstatus(0);
23     }
24     else {
25         USE_SERIAL.print("\nWiFi connected : no ! \n");
26         // ESP.restart();
27     }
28
29     /* init remote debug -----*/
30     Debug.begin("GMESP32");
31
32     /* Install OTA -----*/
33     setupOTA("GM_ESP");
34
35     /* your init code ? -----*/
36 }
37
38 String get_uptime() {
39     /* Return a String of the uptime of the esp */
40     String UPTIME;
41     uptime::calculateUptime();
```

```

42     UPTIME = String(uptime::getMinutes()) +"min and " + String(uptime::getSeconds()) + "sec";
43     return UPTIME;
44 }
45
46 /* Delay can be non compatible with OTA handle !!!!
47 * because ESP32 pauses your program during the delay().
48 * If next OTA request is generated while Arduino is paused waiting for
49 * the delay() to pass, your program will miss that request.
50 * => Use a non-blocking method, using mills() and a delta to decide to read or not. eg */
51 void DoSmtg (int delay){ /* Do stuff here every delay */
52     static uint32_t tick = 0;
53     if ( millis() - tick < delay)
54         return; /* Do nothing */
55     else{
56         USE_SERIAL.println("delay elapsed => we can process !");
57
58         /* your loop code here ? -----*/
59         Debug.printf("Loop : %s\n", get_uptime().c_str());
60     }
61     tick = millis();
62 }
63
64 void loop() {
65     ArduinoOTA.handle(); // gestion OTA => demande ?
66     Debug.handle();
67
68     DoSmtg(10000);
69 }

```

3.2.2 OTA configuration

Même si on ne utilise pas tous, plusieurs paramètres sont à notre disposition pour "régler" le fonctionnement de l'OTA :

1. setPort() : par défaut le port de communication avec OTA est le 3232.
<https://github.com/platformio/platform-espressif32/issues/214>
2. setHostname() : c'est une option importante puisqu'elle permet d'attribuer un nom à l'ESP32.
 Ce nom de machine sera diffusé sur le réseau et récupéré par l'IDE Arduino.
 - Si vous avez beaucoup d'objets sur votre réseau, il sera beaucoup plus facile de l'identifier par son nom que par un identifiant construit automatiquement à partir du numéro de série.
3. setPassword() : permet de demander un mot de passe depuis l'IDE Arduino avant d'exécuter une mise à jour.
4. setPasswordHash() : permet de stocker l'empreinte (le hash) du mot de passe pour plus de sécurité.

<https://passwordsgenerator.net/md5-hash-generator/>

Quatre autres méthodes permettent d'associer des traitements dans les étapes suivantes :

1. `onStart()` : code à exécuter au démarrage de la mise à jour. Nouveauté sur l'ESP32, il est possible de mettre à jour à distance le fichiers de la zone SPIFFS.

C'est très pratique si vous avez besoin de développer une interface HTML pour votre projet.

2. `onEnd()` : idem à la fin
3. `onProgress()` : durant la progression
4. `onError()` : et en cas d'erreur

```
1  #include "ota.h"
2
3  // gestion du temps pour calcul de la durée de la MaJ
4  unsigned long otamillis;
5
6  void setupOTA(const char* nameprefix) {
7      const int maxlen = 40;
8      char fullhostname[maxlen];
9
10     uint8_t mac[6];
11     WiFi.macAddress(mac);
12
13     // Port by default = 3232
14     // ArduinoOTA.setPort(3232); // Syntax if you need to modify
15
16     // Hostname of ESP => as it will appear in arduino IDE too !
17     snprintf(fullhostname, maxlen, "%s-%02x%02x%02x",
18             nameprefix, mac[3], mac[4], mac[5]); // Append MAC byte
19     ArduinoOTA.setHostname(fullhostname);
20
21     // No authentication by default
22     // ArduinoOTA.setPassword("admin");
23     // Password can be set with it's md5 value as well
24     // MD5(admin) = 21232f297a57a5a743894a0e4a801fc3
25     // ArduinoOTA.setPasswordHash("21232f297a57a5a743894a0e4a801fc3");
26
27     ArduinoOTA
28     .onStart([]() { // lancé au début de la MaJ
29         String type;
30         if (ArduinoOTA.getCommand() == U_FLASH)
31             type = "sketch";
32         else // U_SPIFFS
33             type = "filesystem";
34
35         // NOTE: if updating SPIFFS this would be the place to unmount SPIFFS using SPIFFS.end()
36         Serial.println("Start updating " + type);
37         otamillis=millis();
```

```

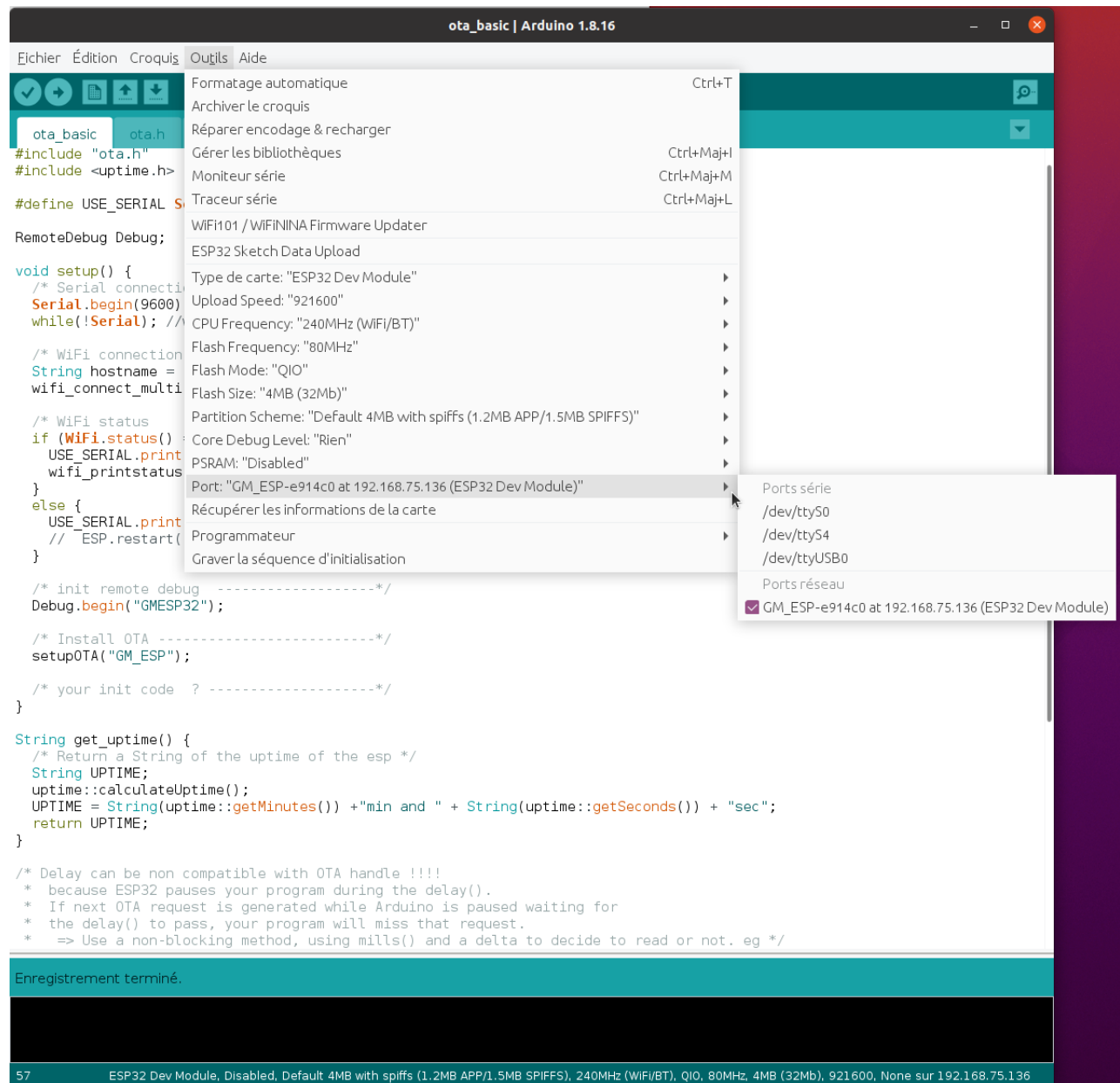
38     })
39     .onEnd([]() {// lancé en fin MaJ
40         Serial.println("\nEnd");
41         Serial.print((millis()-otamillis)/1000.0);
42         Serial.println(" secondes");
43     })
44     .onProgress([](unsigned int progress, unsigned int total) {// lancé lors de la progression de la MaJ
45         Serial.printf("Progress: %u%%\n", (progress / (total / 100)));
46     })
47     .onError([](ota_error_t error) {// En cas d'erreur
48         Serial.printf("Error[%u]: ", error);
49         switch(error) {
50             // erreur d'authentification, mauvais mot de passe OTA
51             case OTA_AUTH_ERROR: Serial.println("OTA_AUTH_ERROR: Auth Failed");
52                                     break;
53             // erreur lors du démarrage de la MaJ (flash insuffisante)
54             case OTA_BEGIN_ERROR: Serial.println("OTA_BEGIN_ERROR: Begin Failed");
55                                     break;
56             // impossible de se connecter à l'IDE Arduino
57             case OTA_CONNECT_ERROR: Serial.println("OTA_CONNECT_ERROR: Connect Failed");
58                                     break;
59             // Erreur de réception des données
60             case OTA_RECEIVE_ERROR: Serial.println("OTA_RECEIVE_ERROR: Receive Failed");
61                                     break;
62             // Erreur lors de la confirmation de MaJ
63             case OTA_END_ERROR: Serial.println("OTA_END_ERROR: Receive Failed");
64                                     break;
65             // Erreur inconnue
66             default: Serial.println("Erreur inconnue");
67         }
68     });
69
70     // Activation fonctionnalité OTA
71     ArduinoOTA.begin();
72
73     Serial.println("OTA Initialized");
74     Serial.print("IP address: ");
75     Serial.println(WiFi.localIP());
76 }

```

4 Uploading OTA

4.1 Uploading from Arduino IDE

Pour qu'OTA soit accessible, il faut bien sûr être sur le même réseau (à moins de mettre en oeuvre une mécanique réseau plus complexe comme la translation d'adresse au niveau du routeur) et normalement vous devez voir apparaître parmi les ports la possibilité "OTA" via un port "réseau" :



Pour mettre à jour le programme, il suffit de sélectionner l'ESP comme port à la place du port série habituel. Puis téléverser le programme comme d'habitude.

4.2 Uploading from the Shell

Avec l'utilitaire Python `espota.py` vous pouvez télécharger (via Internet) en ligne de commande :

<https://github.com/espressif/arduino-esp32/tree/master/tools>

```
python3 espota.py -r -i 192.168.1.5 -f YOUR-FIRMWARE.bin -d -p 3232 -a YOUR-PASSWD
```

Vous aurez éventuellement produit votre firmware depuis l'IDE Arduino :

<https://randomnerdtutorials.com/bin-binary-files-sketch-arduino-ide/>

4.3 Goodbye ... serialxxxx

En passant par le Wifi, vous perdrez la possibilité de déboguer par le port série.

Mais vous avez des bibliothèques comme

- <https://github.com/LennartHennigs/ESPTelnet>
- <https://github.com/JoaoLopesF/RemoteDebug>
- <https://randomnerdtutorials.com/esp32-webserial-library/>

qui permettent de continuer à accéder à l'ESP en y faisant tourner un serveur telnet ou http ou ...

- Dans le cas d'un serveur telnet, il suffit alors de se connecter (avec telnet ou Putty) sur le port 23 de l'adresse IP de l'ESP pour visualiser les messages dans une console déportée sur votre machine.

Vous remarquerez dans le code du fichier "ota_basic.ino" la présence d'un "Debug en ligne" basé sur la bibliothèque de Joao Lopes (RemoteDebug) installable dans arduino IDE par le gestionnaire de bibliothèques.

4.4 TODO : OTA

You have to experiment this code and understand the OTA firmware deployment.

- Most of the time we will keep using the USB for our futures applications ... because I don't have enough battery for everybody :-(